

Manual y Bitácora de Conversaciones

Proyecto IA_BTC_FORECAST

CERO & Gemini Code Assist

28 de octubre de 2025

Resumen

Este documento sirve como una bitácora de las conversaciones mantenidas con el asistente de IA Gemini durante el desarrollo del proyecto `IA_BTC_FORECAST`. Su propósito es documentar las preguntas, respuestas, decisiones de diseño y fragmentos de código generados.

Índice

1. Introducción a Google Colab	3
1.1. Conocimiento sobre la plataforma	3
1.2. Soporte de Lenguajes	3
1.3. Requisitos de Registro	3
2. Integración del Proyecto con Colab y LaTeX	3
2.1. Subir el proyecto a Colab usando Git	3
2.2. Creación de Guía en LaTeX	4
2.3. Corrección de Error de Compilación en LaTeX	4
3. Desarrollo y Herramientas	4
3.1. Revisión y Mejora del script <code>commit.sh</code>	4
3.2. Capacidades de Asistencia en IA con TensorFlow	5
4. Creación de esta Bitácora	5
5. Análisis de la Arquitectura del Modelo	5
5.1. Explicación Detallada de Capas LSTM y Dense	5
5.1.1. La Capa LSTM (Long Short-Term Memory)	5
5.1.2. La Capa Dense	7
5.1.3. Resumen del Flujo de Información	7
6. Flujo de Trabajo en Google Colab	7
6.1. Inicio de un Nuevo Notebook	7
6.2. Clonado de Repositorios (Público vs. Privado)	8
6.3. Error Común: URL de Navegación vs. URL de Clonado	9
6.4. Visualización del Contenido del Repositorio	9
6.5. Activación y Verificación de GPU	10
6.5.1. Paso 1: Activar la GPU	10
6.5.2. Paso 2: Verificar que la GPU está activa	10

7. Verificación del Entorno en Google Colab	11
7.1. Resultado de la Verificación de GPU	11
8. Aceleración del Entrenamiento con GPU	12
8.1. Adaptación del Script de Entrenamiento	12
8.2. Conceptos Clave: Épocas y Tamaño de Lote	12
8.2.1. Épocas (Epochs)	12
8.2.2. Tamaño del Lote (Batch Size)	13
8.2.3. Resumen del Flujo	13
9. Optimización del Entrenamiento	13
9.1. Determinación del Número Ideal de Épocas	13
9.1.1. El Problema: Subentrenamiento vs. Sobreentrenamiento	13
9.1.2. La Solución: El Callback <code>EarlyStopping</code> de Keras	14

1. Introducción a Google Colab

1.1. Conocimiento sobre la plataforma

Usuario:

Gemini, ¿conoces una plataforma llamada "Google Colab"?

Gemini:

Sí, por supuesto. Google Colaboratory, o Colab", es un servicio en la nube gratuito de Google que permite escribir y ejecutar código Python en el navegador. Es un entorno de Jupyter Notebook sin configuración, con acceso a hardware gratuito (GPUs y TPUs), ideal para Machine Learning.

1.2. Soporte de Lenguajes

Usuario:

¿Esa plataforma solo sirve para programar en Python?

Gemini:

Principalmente sí, está optimizado para Python. Sin embargo, también se pueden ejecutar comandos de shell (usando `!`) y, con algo de configuración, se pueden instalar kernels para otros lenguajes como R o Julia.

1.3. Requisitos de Registro

Usuario:

¿Para usar esa plataforma hay que registrarse en la misma?

Gemini:

Sí, pero no es un registro separado. Solo necesitas una cuenta de Google. Si ya usas Gmail o Google Drive, puedes usar esa misma cuenta. Tus notebooks se guardan directamente en tu Google Drive.

2. Integración del Proyecto con Colab y LaTeX

2.1. Subir el proyecto a Colab usando Git

Usuario:

¿Cómo puedo subir mi proyecto IA_BTC_FORECAST a Google Colab usando Git?

Gemini:

El proceso es sencillo:

1. Clonar el repositorio: En una celda de Colab, ejecuta:

```
1 !git clone https://github.com/ceroiglesias/IA_BTC_FORECAST.git
2
```

2. Cambiar de directorio: Usa el comando mágico:

```
1 %cd IA_BTC_FORECAST
2
```

3. Instalar dependencias: Ejecuta:

```
1 !pip install -r requirements.txt
2
```

4. Ejecutar scripts: Llama a tus scripts con:

```
1 !python nombre_script.py
2
```

2.2. Creación de Guía en LaTeX

Usuario:

¿Podrías vaciarme toda esta información que me acabas de dar en un archivo latex?

Gemini:

Claro. Generé un documento `.tex` completo con toda la información sobre Google Colab, sus características y la guía para subir tu proyecto. El archivo incluye un preámbulo bien configurado para formatear el texto y los bloques de código.

2.3. Corrección de Error de Compilación en LaTeX

Usuario:

Tenemos un error al compilar el archivo latex. [Se adjunta log de error].

Gemini:

El error `Invalid UTF-8 byte sequence` se debe a que el paquete `listings` no está interpretando correctamente los caracteres con tilde dentro de los comentarios del código. La solución es añadir la opción `literate` a la configuración de `lstdefinestyle` para que LaTeX reemplace los caracteres especiales (como `'ó'`, `'á'`, `'ñ'`) por sus comandos correspondientes (como `{ó}`, `{á}`, `{ñ}`).

3. Desarrollo y Herramientas

3.1. Revisión y Mejora del script `commit.sh`

Usuario:

Revisa el script `commit.sh` que te muestro en el contexto.

Gemini:

El script es excelente. Propuse algunas mejoras para aumentar su robustez y claridad:

- Usar un "here document" para la función de ayuda, haciéndola más mantenible.
- Hacer que el mensaje de commit sea obligatorio.
- Añadir una comprobación interactiva para `flake8`: si encuentra errores, pregunta al usuario si desea continuar.

- Comprobar si hay cambios pendientes antes de intentar hacer el commit para evitar errores.

Se proporcionó una versión refactorizada del script con estos cambios.

3.2. Capacidades de Asistencia en IA con TensorFlow

Usuario:

¿Estás en capacidad de ayudarme a desarrollar un script IA en python que utilice la librería Tensor Flow?

Gemini:

¡Por supuesto que sí! Es una de mis especialidades. Puedo ayudarte a escribir código desde cero, explicar conceptos de TensorFlow/Keras, depurar tus scripts o trabajar contigo para optimizar tu modelo `IA_BTC_FORECAST`.

4. Creación de esta Bitácora

Usuario:

Perfecto Gemini, pero para no perder esta y futuras conversaciones quiero que crees un archivo en formato latex con esta conversación y luego guardalo en el directorio `/home/cero/MEGA/VSCODE/Bin` el nombre de `"manual.tex"`. Mas adelante, cuando te pida que guardes una conversación, la anexas a ese archivo (`manual.text`) en formato latex.

Gemini:

¡Excelente idea! He creado este documento, `manual.tex`, en la ruta especificada. Contiene un resumen de nuestra conversación hasta la fecha. A partir de ahora, cuando me lo indiques, añadiré nuestras nuevas interacciones a este mismo archivo para mantener un registro completo.

5. Análisis de la Arquitectura del Modelo

5.1. Explicación Detallada de Capas LSTM y Dense

Usuario:

*Ya compilé el archivo `manual.tex` sin errores. Ahora explícame en detalle las capas **LSTM** y **Dense** de mi modelo actual.*

Gemini:

¡Genial que el archivo `manual.tex` haya compilado correctamente! Ahora tenemos una excelente bitácora para nuestro trabajo.

*Vamos a sumergirnos en el corazón de tu modelo de predicción. Las capas **LSTM** y **Dense** son los ladrillos fundamentales con los que estás construyendo tu red neuronal.*

5.1.1. La Capa LSTM (Long Short-Term Memory)

*Piensa en la capa **LSTM** como el **cerebro con memoria** de tu modelo. Su superpoder es entender **secuencias y contexto a lo largo del tiempo**. Esto es absolutamente crucial para tu proyecto, porque el precio futuro del Bitcoin no depende solo del precio de ayer, sino de*

la tendencia, el impulso y los patrones que se han formado durante los últimos días, semanas o meses.

¿Cómo funciona? (Analogía) Imagina que estás leyendo una novela. Para entender la trama, no solo lees la palabra actual, sino que recuerdas a los personajes y los eventos que ocurrieron en capítulos anteriores.

- **RNN Simple (Red Neuronal Recurrente):** Sería como una persona con muy mala memoria a corto plazo. Recuerda la frase anterior, pero olvida rápidamente lo que pasó al principio del libro. Esto se conoce como el "problema del gradiente desvaneciente".
- **LSTM:** Es como un lector atento. Tiene un cuaderno de notas.^{es}pecial (el **estado de la celda** o cell state) donde apunta las cosas importantes de la trama. Además, tiene tres "supervisores."^o **compuertas** (gates) que gestionan ese cuaderno:
 1. **Compuerta de Olvido (Forget Gate):** Decide qué información del cuaderno ya no es relevante y debe ser borrada. Por ejemplo: ".^{Este} personaje secundario murió, ya no necesito recordarlo". En tu caso: "La pequeña fluctuación de precio de hace 30 días ya no es importante para la tendencia actual".
 2. **Compuerta de Entrada (Input Gate):** Decide qué nueva información de la palabra que está leyendo es lo suficientemente importante como para añadirla al cuaderno. Por ejemplo: "¡Un nuevo villano ha aparecido! Esto es clave, lo apunto". En tu caso: ".^{El} precio acaba de romper una resistencia importante, jesto es una señal clave y debo recordarla!".
 3. **Compuerta de Salida (Output Gate):** Decide qué parte del cuaderno es relevante para la acción inmediata que debe tomar (la predicción). Por ejemplo: "Basado en mis notas sobre el héroe y el villano, creo que el próximo capítulo será una pelea". En tu caso: "Basado en la tendencia alcista que he memorizado y la reciente ruptura de resistencia, mi predicción para el siguiente día será un valor más alto".

Parámetros Claves en tu Modelo

- **units=50:** Esto significa que tu capa LSTM tiene 50 "neuronas."^o inidades de memoria". Cada una de estas unidades mantiene su propio cuaderno de notas"(su propio estado de celdas y estado oculto) para aprender un patrón o aspecto diferente de los datos. Más unidades pueden capturar patrones más complejos pero también aumentan el riesgo de sobreajuste (memorizar los datos de entrenamiento en lugar de aprender a generalizar).
- **return_sequences=True:** Este es un parámetro vital cuando apilas varias capas LSTM. Le dice a la capa: "No me des solo tu conclusión final después de ver los 60 días; dame tus 'pensamientos' o estado oculto para cada uno de los 60 días". Esto permite que la siguiente capa LSTM reciba una secuencia completa y pueda seguir construyendo sobre los patrones que la capa anterior encontró. Si fuera la última capa LSTM antes de una capa **Dense**, normalmente se pondría en **False**.
- **input_shape=(X_train.shape[1], 1):** Esto es solo para la primera capa. Le dice al modelo cómo es una muestra de entrada. En tu caso, es (60, 1), lo que significa que cada muestra es una secuencia de 60 pasos de tiempo (días) con 1 característica por paso (el precio de cierre).

5.1.2. La Capa Dense

Después de que las capas *LSTM* han hecho su trabajo de entender la secuencia y extraer patrones temporales, la capa *Dense* entra en acción. Piensa en ella como el **cerebro que toma decisiones**. A diferencia de la *LSTM*, no tiene memoria de secuencias. Su trabajo es tomar la información que le pasan (sin importar el orden) y encontrar las mejores combinaciones para llegar a una conclusión.

¿Cómo funciona? Cada neurona en una capa *Dense* está conectada a **todas** las neuronas de la capa anterior. Recibe todas las salidas, las pondera (les da más importancia a unas que a otras) y, basándose en esas ponderaciones, produce una salida.

Su Rol en tu Modelo En tu arquitectura, las capas *Dense* cumplen dos funciones:

1. **Capas Dense Intermedias (*units*=25):** Estas capas actúan como un "procesador de conclusiones". La última capa *LSTM* le pasa un resumen muy rico y comprimido de todos los patrones temporales que encontró en los 60 días. La capa *Dense* toma este resumen y aprende a combinar las diferentes piezas de información. Es como si dijera: ".ºk, la *LSTM* me dice que hay una tendencia alcista (patrón A) pero también una pequeña señal de reversión a corto plazo (patrón B). ¿Cómo combino A y B para refinar mi predicción?". Ayuda al modelo a aprender relaciones no lineales más complejas sobre los patrones ya extraídos.
2. **Capa Dense de Salida (*units*=1):** Esta es la capa final, la que da la respuesta. Tiene una sola neurona porque tu objetivo es predecir un solo valor: el precio del Bitcoin para el siguiente día. Esta neurona toma toda la información procesada por las capas anteriores y la condensa en un único número, que es la predicción final del modelo.

5.1.3. Resumen del Flujo de Información

1. **Entrada:** Una ventana de 60 días de precios de BTC.
2. **Capas LSTM:** Procesan la secuencia día a día, recordando patrones y tendencias a largo plazo y pasando una secuencia enriquecida a la siguiente capa.
3. **Capa Dense(25):** Recibe el resumen final de las *LSTM* y lo procesa para encontrar combinaciones complejas entre los patrones detectados.
4. **Capa Dense(1):** Recibe la información de la capa anterior y la concentra en una única salida numérica: la predicción del precio para el día 61.

Espero que esta explicación detallada te ayude a entender mejor cómo "piensa" tu modelo. ¡ Es una arquitectura muy potente para el problema que estás resolviendo!

6. Flujo de Trabajo en Google Colab

6.1. Inicio de un Nuevo Notebook

Usuario:

Una vez que se entra a "Google Colab" hay que iniciar un nuevo notebook. Ciertamente?

Gemini:

¡Correcto! Una vez que entras a Google Colab, lo más común es iniciar un nuevo notebook para empezar a trabajar.

Cuando visitas colab.research.google.com, generalmente te aparece una ventana emergente que te da varias opciones:

- **Crear un notebook nuevo:** La opción más directa para un proyecto desde cero.
- **Abrir un notebook reciente:** Para continuar con tu trabajo.
- **Abrir desde Google Drive:** Si tienes notebooks guardados allí.
- **Abrir desde GitHub:** Puedes pegar la URL de un repositorio o un notebook específico.
- **Subir un archivo:** Si tienes un archivo `.ipynb` en tu ordenador.

Para nuestro proyecto `IA_BTC_FORECAST`, el flujo de trabajo ideal es exactamente ese:

1. Iniciar un notebook nuevo.
2. En la primera celda, clonar tu repositorio de GitHub con:

```
1
```

De esta manera, mantienes un entorno limpio y reproducible cada vez que trabajas.

6.2. Clonado de Repositorios (Público vs. Privado)

Usuario:

Creo que para clonar un repositorio de github, antes hay que hacerlo público?

Gemini:

¡Esa es una excelente pregunta! No, no es estrictamente necesario. Puedes clonar tanto repositorios públicos como privados, pero el procedimiento es diferente:

1. Repositorios Públicos:

- Cualquiera puede clonarlos sin necesidad de autenticarse.
- Este es el caso de tu proyecto `IA_BTC_FORECAST`, por eso el comando:

```
2
```

funciona directamente en Colab.

2. Repositorios Privados:

- Necesitas tener permisos de acceso y debes autenticarte.
- Las formas más comunes son usando un **Token de Acceso Personal (PAT)** con la URL HTTPS, o configurando una **clave SSH**.
- Por ejemplo, para clonar usando un PAT, la URL se vería así:

```
1 !git clone https://<TOKEN>@github.com/usuario/repositorio.git
```

```
2
```

- Alternativamente, puedes configurar una clave SSH en Colab, pero esto es un poco más avanzado y requiere pasos adicionales.

En resumen, para tu proyecto actual no tienes que preocuparte porque es público. Pero si en el futuro trabajas con un repositorio privado, solo necesitarás autenticarte para poder clonarlo.

6.3. Error Común: URL de Navegación vs. URL de Clonado

Usuario:

Le doy el siguiente camino: `https://github.com/cero97-ctrl/repositorio/tree/main/Binance/Python`. Pero me dice que no encuentra el repositorio.

Gemini:

¡Entiendo perfectamente el problema! Es un error muy común. La URL que estás usando es la URL de **navegar** por una subcarpeta en la web de GitHub, no para clonar.

El comando `git clone` necesita la URL principal del repositorio, que siempre descarga el proyecto completo.

La Solución

1. **Usa la URL correcta para clonar.** Para tu repositorio, es:

```
1 https://github.com/cero97-ctrl/repositorio.git
2
```

2. **Clona el repositorio completo:**

```
1 !git clone https://github.com/cero97-ctrl/repositorio.git
2
```

3. **Navega a la carpeta deseada:** Una vez clonado, puedes moverte a la subcarpeta que necesitas:

```
1 import os
2 os.chdir('repositorio/Binance/Python')
```

6.4. Visualización del Contenido del Repositorio

Usuario:

¿Cómo muestro el contenido del repositorio?

Gemini:

¡Claro! Una vez que has clonado el repositorio, este se descarga en una nueva carpeta. Para ver su contenido desde Colab, puedes usar el comando `ls` de Linux, prefijado con un signo de exclamación (!).

Pasos para ver el contenido

1. **Clona el repositorio (si no lo has hecho ya):**

```
1 !git clone https://github.com/cero97-ctrl/repositorio.git
2
```

2. **Lista el contenido de la nueva carpeta:**

```
1 !ls repositorio
2
```

3. **Para una vista más detallada (con permisos, tamaños y archivos ocultos):**

```
1 !ls -lha repositorio
2
```

Flujo de trabajo alternativo También puedes 'entrar' en el directorio del repositorio con:

```
1 %cd repositorio
2
```

y luego ejecutar `!ls -lha` para listar el contenido del directorio actual. Esto es útil si vas a ejecutar varios comandos dentro de esa carpeta.

6.5. Activación y Verificación de GPU

Usuario:

¿Cómo puedo activar y verificar el uso de la GPU en este notebook de Colab?

Gemini:

¡Claro! Activar la GPU en Colab es uno de sus puntos más fuertes y es muy sencillo. Aquí te explico cómo hacerlo y cómo comprobar que está funcionando

6.5.1. Paso 1: Activar la GPU

El entorno de ejecución de Colab usa una CPU por defecto. Para cambiar a una GPU, sigue estos pasos:

1. Ve al menú superior y haz clic en **Entorno de ejecución** (Runtime).
2. En el menú desplegable, selecciona **Cambiar tipo de entorno de ejecución** (Change runtime type).
3. Se abrirá una ventana. Busca el menú desplegable llamado **Acelerador por hardware** (Hardware accelerator).
4. Selecciona **T4 GPU** (o la opción de GPU que aparezca).
5. Haz clic en **Guardar** (Save).

Importante: Este cambio reiniciará tu entorno de ejecución, por lo que tendrás que volver a ejecutar las celdas anteriores (como la de clonar el repositorio o instalar dependencias).

6.5.2. Paso 2: Verificar que la GPU está activa

Una vez que el entorno se haya reiniciado, puedes verificar que la GPU está disponible de dos maneras muy sencillas.

Método 1: Con un comando de shell Este es el método más rápido y común. Ejecuta la siguiente celda:

```
1 !nvidia-smi
```

Si la GPU está activa, verás una tabla con información detallada sobre la GPU asignada (generalmente una Tesla T4). Si obtienes un error como `nvidia-smi: command not found`, significa que la GPU no está activada.

Método 2: Con TensorFlow También puedes pedirle a TensorFlow que liste los dispositivos GPU que detecta. Ejecuta esta celda:

```
1 import tensorflow as tf
2 print(tf.config.list_physical_devices('GPU'))
```

Si la GPU está funcionando, la salida será una lista que contiene información sobre el dispositivo GPU, algo como: `[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]`. Si la lista está vacía (`[]`), TensorFlow no está detectando la GPU.

7. Verificación del Entorno en Google Colab

7.1. Resultado de la Verificación de GPU

Usuario:

He activado y verificado la GPU en mi cuenta de "Google Colab" la respuesta fué la que te muestro en el contexto.

Gemini:

¡Perfecto! El resultado que me muestras es exactamente lo que esperábamos ver. Confirma que Google Colab te ha asignado correctamente una GPU y que tu entorno está listo para el entrenamiento acelerado.

Analicemos brevemente la salida que recibiste:

```

1 +-----+
2 | NVIDIA-SMI 550.54.15                Driver Version: 550.54.15      CUDA
   | Version: 12.4                  |
3 +-----+-----+-----+-----+-----+-----+
4 | GPU   Name                               Persistence-M | Bus-Id        Disp.A |
   | Volatile Uncorr. ECC |
5 | Fan    Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-
   | Util    Compute M. |
6 |                               |                      |
   |          MIG M. |
7 +=====+=====+=====+=====+=====+=====+
8 |    0   Tesla T4                               Off | 00000000:00:04.0 Off |
   |               0 |
9 | N/A     56C    P8              12W / 70W |      0MiB / 15360MiB |      0%
   |      Default |
10 |                               |                      |
   |          N/A |
11 +-----+-----+-----+-----+-----+-----+

```

Listing 1: Salida del comando `!nvidia-smi`

Puntos clave de esta salida:

- **GPU Name:** Tienes asignada una *Tesla T4*, una GPU excelente para tareas de Machine Learning.
- **Memory-Usage:** Dispones de casi 16 GB de memoria de video (*15360MiB*), que está completamente libre para ser usada.
- **GPU-Util:** El uso de la GPU es del *0%*, lo que indica que está lista y esperando para procesar tu código.

8. Aceleración del Entrenamiento con GPU

8.1. Adaptación del Script de Entrenamiento

Usuario:

Ahora que la GPU está activa, ¿cómo adapto mi script de entrenamiento para que la use?

Gemini:

*¡Esa es una excelente pregunta! Y la mejor noticia es que, al usar un framework como TensorFlow/Keras, ****prácticamente no necesitas hacer ningún cambio en tu código.*****

TensorFlow está diseñado para detectar y utilizar automáticamente una GPU compatible con CUDA cuando está disponible. Cuando llamas a `model.fit()`, TensorFlow se encarga de:

- 1. Mover el modelo a la GPU.*
- 2. Enviar los lotes (batches) de datos de entrenamiento a la memoria de la GPU.*
- 3. Realizar todos los cálculos intensivos (como la propagación hacia adelante y la retropropagación) directamente en la GPU.*

Tu script `model_forecast.py` se beneficiará de la aceleración por hardware sin modificaciones. La forma más sencilla de verificar que la GPU está trabajando durante el entrenamiento es ejecutar el comando `!nvidia-smi` en otra celda mientras `model.fit()` está en ejecución. Verás que el uso de memoria (Memory-Usage) y el porcentaje de utilización (GPU-Util) aumentan considerablemente.

8.2. Conceptos Clave: Épocas y Tamaño de Lote

Usuario:

Explícame qué son las "épocas" (epochs) y el "tamaño del lote" (batch size) en 'model.fit'.

Gemini:

¡Claro! Son dos de los hiperparámetros más importantes en el entrenamiento de redes neuronales. Usaremos una analogía: el modelo es un estudiante y el conjunto de datos es su libro de texto.

8.2.1. Épocas (Epochs)

*Una **época** representa una pasada completa del modelo a través de **todo el conjunto de datos de entrenamiento**.*

- **Analogía:** Una época es como si el estudiante leyera el libro de texto completo, de principio a fin, una vez.
- **Propósito:** Al igual que un estudiante necesita repasar, el modelo revisa los datos múltiples veces (en múltiples épocas) para aprender mejor los patrones y reducir su error.
- **En tu código:** `epochs=3` significa que el modelo "leerá" todo el conjunto de datos de entrenamiento 3 veces.
- **Riesgos:**
 - **Pocas épocas:** El modelo no aprende lo suficiente (underfitting).

- **Demasiadas épocas:** El modelo puede empezar a memorizar los datos de entrenamiento en lugar de generalizar, fallando con datos nuevos (overfitting).

8.2.2. Tamaño del Lote (Batch Size)

El **tamaño del lote** es el número de muestras de entrenamiento que el modelo procesa antes de realizar una actualización de sus parámetros internos (pesos).

- **Analogía:** El estudiante no puede asimilar el libro entero de una vez. Estudia por partes (capítulos o secciones). El tamaño del lote es el tamaño de esa "sección de estudio".
- **Proceso:** El conjunto de datos se divide en lotes. El modelo procesa un lote, calcula el error, ajusta sus pesos, y luego pasa al siguiente lote. Cuando ha procesado todos los lotes, ha completado una época.
- **En tu código:** `batch_size=1` es un caso especial. El modelo ajusta sus pesos después de ver cada muestra individual. Es computacionalmente ineficiente y puede ser inestable, aunque a veces ayuda a evitar mínimos locales. Un valor común y más eficiente es 32.
- **Riesgos:**
 - **Batch size pequeño:** Aprendizaje lento y ruidoso, no aprovecha la GPU.
 - **Batch size grande:** Aprendizaje rápido y estable, pero a veces puede converger a una solución menos óptima.

8.2.3. Resumen del Flujo

Si tienes 1000 muestras, `epochs=5` y `batch_size=100`, el modelo hará lo siguiente:

1. Dividirá las 1000 muestras en 10 lotes de 100.
2. En cada época, procesará los 10 lotes, ajustando sus pesos 10 veces.
3. Repetirá este proceso 5 veces (5 épocas).

9. Optimización del Entrenamiento

9.1. Determinación del Número Ideal de Épocas

Usuario:

¿Cómo puedo saber cuál es el número ideal de épocas para mi modelo?

Gemini:

¡Esa es una de las preguntas más importantes en Machine Learning! No existe un número mágico, pero la técnica estándar para encontrar el punto óptimo es usar **Parada Temprana** (Early Stopping).

9.1.1. El Problema: Subentrenamiento vs. Sobreentrenamiento

El objetivo es entrenar lo suficiente para que el modelo aprenda los patrones (subentrenamiento), pero no tanto como para que empiece a memorizar los datos de entrenamiento y falle con datos nuevos (sobreentrenamiento).

Para detectar esto, monitorizamos dos métricas durante el entrenamiento:

- **Pérdida de Entrenamiento (Training Loss):** El error sobre los datos que el modelo usa para aprender. Casi siempre disminuye con más épocas.
- **Pérdida de Validación (Validation Loss):** El error sobre un conjunto de datos que el modelo no ha visto. Esta es la métrica clave.

El número ideal de épocas es el punto donde la pérdida de validación alcanza su valor mínimo, justo antes de empezar a subir de nuevo.

9.1.2. La Solución: El Callback EarlyStopping de Keras

En lugar de adivinar un número de épocas, le decimos a Keras que monitorice la pérdida de validación y detenga el entrenamiento automáticamente cuando deje de mejorar.

Implementación Práctica:

1. Importar el callback:

```
1 from tensorflow.keras.callbacks import EarlyStopping
2
```

2. Configurar el callback: Se definen tres parámetros clave:

```
1 # monitor='val_loss': La métrica a vigilar.
2 # patience=10: Cuántas épocas esperar sin mejora antes de parar.
3 # restore_best_weights=True: Devuelve el modelo con los pesos de la
   mejor época.
4 early_stopping = EarlyStopping(monitor='val_loss', patience=10,
   restore_best_weights=True)
5
```

3. Añadirlo a model.fit(): Se pasa el callback y se reserva una parte de los datos para la validación.

```
1 history = model.fit(X_train, y_train, epochs=200, batch_size=32,
   validation_split=0.2, callbacks=[early_stopping])
2
```

Con este método, puedes poner un número alto de épocas (ej. 200) con la seguridad de que el entrenamiento se detendrá en el momento óptimo, evitando el sobreentrenamiento y ahorrando tiempo de cómputo.